

Building Research Software with AI

Summer School in AI & Applied Economics

Fabian Roeben

ETH Zürich

September 2026

Plan for today

1. Why software belongs in research design
2. When to use, partner, or build
3. How to move from a prototype to specified software
4. A worked case: TailorMadeTimes
5. How to test software and validate the research
6. How to deploy, monitor, and preserve a research system

Software as research infrastructure

Empirical research is constrained by where data can be observed or generated

- ▶ Researchers either observe existing systems or create environments that generate new data.
- ▶ Observational work is limited by what institutions, platforms, APIs, and administrative systems record and expose.
- ▶ Experiments offer more control, but the setting still limits feasible treatments and measurements.
- ▶ Some questions are difficult because the required environment does not exist or researchers cannot access it.

Empirical settings trade off control, realism, access, and measurement

- ▶ **Observational settings:** natural behavior and potentially large scale, but limited control over what is recorded or exposed.
- ▶ **Field settings with external partners:** natural users and institutions, but shared control over access, timing, implementation, and measurement.
- ▶ **Lab and survey settings:** high control over treatment and measurement, but a narrower and often less natural interaction space.

Every setting gives up something: control, realism, access, or measurement.

Software can create new data-generating and measurement environments

Researchers can build software for:

- ▶ browser or mobile instrumentation and new behavioral traces;
- ▶ custom participant and data-collection interfaces;
- ▶ annotation and coding systems;
- ▶ repeated behavioral or experimental environments;
- ▶ automated ingestion and data-processing pipelines.

Custom software can support richer interaction than a standard survey while giving researchers more control over measurement and repeated behavior.

A research question can require an environment that does not yet exist

- ▶ **Question:** How do people want news to be presented, and does customization improve matching?
- ▶ Existing choices bundle source, event, tone, ideology, complexity, and other attributes.
- ▶ The project required an environment that could vary presentation while holding the underlying news event fixed.

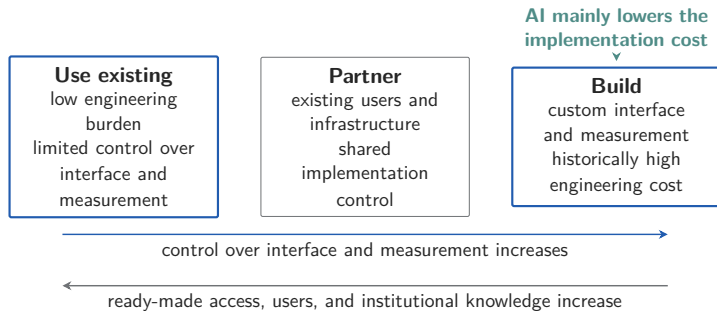


AI changes the cost of building research infrastructure

- ▶ Bespoke research software has historically required substantial developer time and specialized expertise.
- ▶ Coding agents reduce the fixed cost of implementation, debugging, and maintenance.
- ▶ Some research environments that were impractical for small academic teams may now be feasible.
- ▶ Reliability, security, deployment, and maintenance still require deliberate choices.

Choosing what to build

How can researchers access the infrastructure they need?



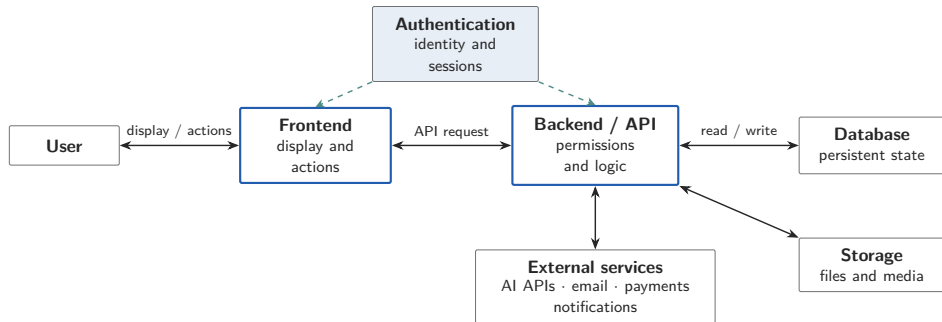
The choice determines who controls the interface, measurement, users, and implementation.

Building it yourself shifts the constraint

- ▶ Existing platforms may already have users who participate for their own reasons. A researcher-built application starts without that user base.
- ▶ With **organic use**, people have their own reason to use the product. With **research-induced use**, researchers recruit and usually compensate them.
- ▶ Building increases control but may create more dependence on recruitment, incentives, engagement, and access.
- ▶ Selection into adoption and continued use becomes part of the realized data-generating process.

AI lowers engineering costs. It does not supply users or give them a reason to participate.

Most applications use the same basic components



Components have different responsibilities and communicate through defined interfaces.

Frontend and backend have different responsibilities

Frontend

- ▶ Runs in a browser or on the user's device.
- ▶ Displays information and collects actions through forms, feeds, buttons, charts, and mobile screens.

Backend

- ▶ Runs in trusted infrastructure rather than in the user's client.
- ▶ Implements permissions, database operations, research logic, external API calls, and background jobs.

Security rule: Sensitive logic and unrestricted credentials belong outside the client.

The database stores persistent application state

- ▶ A database stores **persistent structured state** such as users, assignments, items, exposures, and events.
- ▶ Persistent state matters whenever users return or interact over time.
- ▶ In research applications, database events often become the raw material for the final dataset.
- ▶ The schema, meaning the structure of tables and relationships, determines which constraints the system can enforce.

APIs, identity, and permissions define system boundaries

- ▶ **API:** How does one component request information or an action from another?
- ▶ **Authentication:** Who is making the request?
- ▶ **Authorization:** What may that user do?
- ▶ **Deployment:** Where does the application run for real users?

`GET /articles/123 → return article data`

`POST /bookmark → save for the authenticated user`

Where does the relevant state live, and who is allowed to change it?

From vibe coding to specification

Coding is not software engineering

- ▶ **Coding** means producing instructions a computer executes.
- ▶ **Software engineering** means making a system understandable, maintainable, testable, and reliable as it grows.
- ▶ Coding agents have made implementation dramatically cheaper.
- ▶ They have not removed the need to decide how the system should behave and how it should be structured.

The difficult part is shifting from writing code to specifying and checking behavior.

What coding agents are already good at

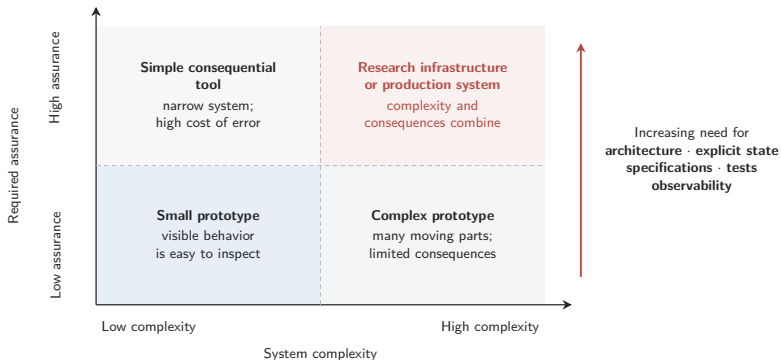
- ▶ Scaffold applications and components.
- ▶ Connect APIs, databases, and external services.
- ▶ Debug and refactor existing systems.
- ▶ Generate tests and explain unfamiliar code.
- ▶ Apply repeated changes consistently across a project.

Why vibe coding works

- ▶ Natural-language iteration with immediate feedback can be highly productive.
- ▶ The user interface itself often provides a fast feedback signal.
- ▶ Vibe coding works especially well when the system is small and correctness is easy to observe.
- ▶ For prototypes and internal tools, this may be entirely sufficient.

The term was introduced by Karpathy (2025); see Sarkar and Drosos (2025).

Where vibe coding becomes risky



Visible progress is a weak standard when the software determines the research data.

From “make this work” to a feature specification

Feature: Users can bookmark an item.

- ▶ **Behavior:** Clicking saves the item.
- ▶ **State:** The bookmark belongs to the authenticated user and item.
- ▶ **Edge case:** Clicking again removes it.
- ▶ **Persistence:** The bookmark survives logout and login.
- ▶ **Acceptance criteria:** It persists correctly and is visible only to its owner.

Acceptance criteria make software tasks testable

- ▶ Define observable success before implementation.
- ▶ State the required behavior, persistent state, and important edge cases.
- ▶ Turn the criteria into manual checks or automated tests.
- ▶ Ask the agent to restate the specification and identify unresolved assumptions before writing code.

Software layers last longer than product names

Layer	One reasonable default
Web application	TypeScript and Next.js
Mobile application	Expo and React Native
<i>Backend, database, and auth</i>	
Managed backend and database	Supabase or Convex
Authentication	Built-in auth or Clerk where needed
Background services	Railway
Hosting and mobile release	Vercel and Expo EAS

These are examples, not course requirements. Choose mainstream, documented tools that a small research team can maintain.

Build what is specific to the research

- ▶ Use managed services for authentication, databases, hosting, and other generic components.
- ▶ Write custom code for research logic, participant workflows, measurement, and treatment delivery.
- ▶ Add background processes or data pipelines only where the project requires them.
- ▶ Managed services reduce operational work but do not remove responsibility for data protection or correctness.

Use managed services for generic components. Spend custom effort on the research logic.

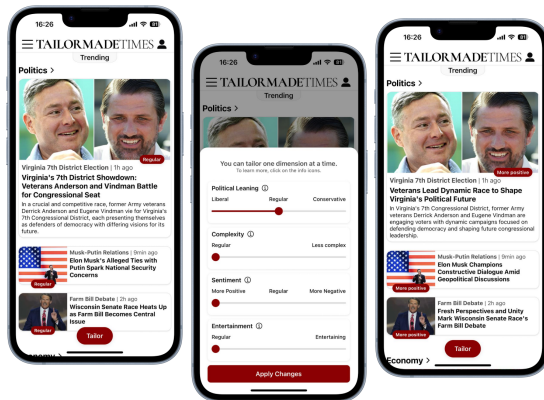
Case study: TailorMadeTimes

Varying news presentation while holding the underlying event fixed

Why this question required a new environment

- ▶ News consumption bundles coverage, source, tone, ideology, complexity, and other attributes.
- ▶ Existing market choices therefore do not cleanly reveal preferences over presentation alone.
- ▶ A conventional survey could elicit stated preferences, but would not reproduce repeated naturalistic news consumption.
- ▶ The research needs an environment that varies presentation while holding the underlying event fixed.

TailorMadeTimes



Users customize presentation while the underlying news event stays fixed.
Source: Chopra et al. (2025).

Same event, different presentation



More negative



Facts only

Both screens describe the same event. Only the presentation changes.
Source: Chopra et al. (2025).

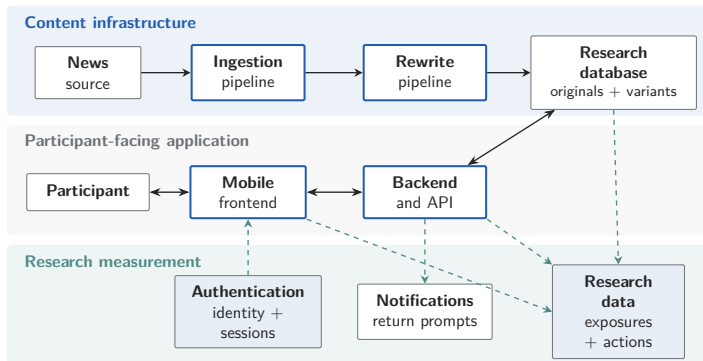
Define what must remain true

Examples of **software invariants**:

- ▶ each participant has one persistent treatment assignment;
- ▶ customization changes presentation, not article selection;
- ▶ control users cannot access customization;
- ▶ only treatment-appropriate options are available;
- ▶ relevant exposures and interactions are recorded.

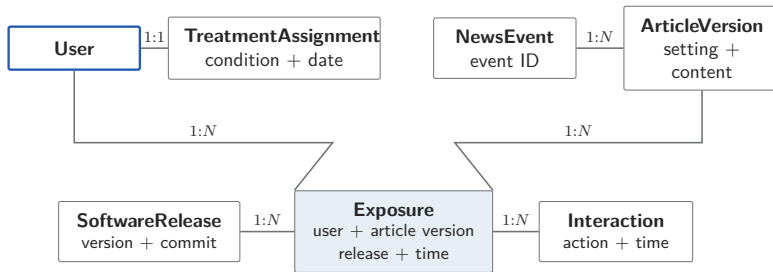
An invariant is a property of the research system that must remain true.

The system behind the app



The participant sees an app; the study depends on the full system behind it.

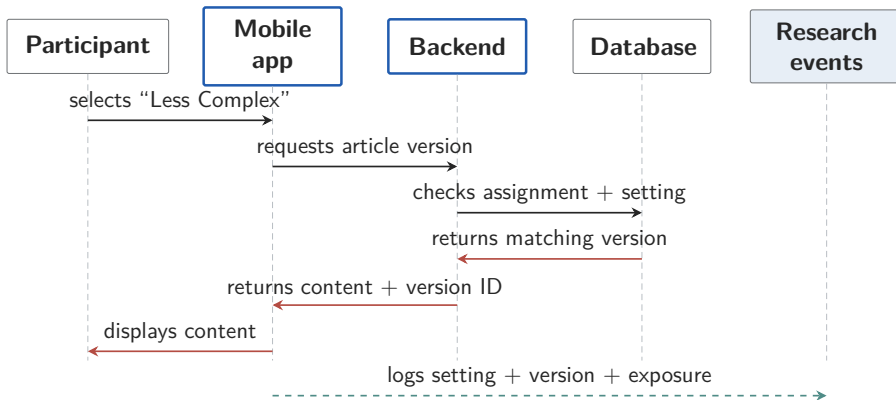
Data models are part of the research design



The exposure record joins **who saw what**, under **which software release**, and **when**.

The schema determines which relationships and constraints the system can enforce.

One feature from click to database



A visible click crosses several components before it becomes research data.
The observation exists only after the full path succeeds.

How I would specify this feature to a coding agent

- ▶ **Goal:** Implement the Less Complex selection.
- ▶ **Invariant:** The underlying event cannot change.
- ▶ **State and access:** The choice persists, and control users cannot request it.
- ▶ **Instrumentation:** Record the setting and article version for each exposure.
- ▶ **Acceptance criteria:** The correct version appears, the event is unchanged, access rules hold, and reload behavior is correct.

Before implementation, ask the agent to identify the relevant components and explain where the change belongs.

Where AI helps and where judgment enters

The agent can help implement

- ▶ user interfaces and API connections;
- ▶ repetitive database logic;
- ▶ tests, debugging, and refactoring.

The researcher must decide

- ▶ architecture and data model;
- ▶ treatment rules and invariants;
- ▶ privacy boundaries and acceptance criteria.

A software bug can change the data-generating process

- ▶ An invariant states how the research system must behave.
- ▶ Violating one can alter treatment, measurement, selection, attrition, or timing.
- ▶ “The app ran” is therefore not a sufficient correctness criterion.
- ▶ A bug can be a protocol deviation even when participants never see an error message.

Software correctness can become part of identification.

Software failures have research consequences

Software failure	Research consequence
Treatment reassigned	Contamination
Wrong treatment version shown	Incorrect treatment exposure
Missing exposure records	Exposure measurement error
Treatment-specific crashes	Differential attrition
Incorrect timestamps	Wrong sequencing
State or cache leakage	Contamination or privacy breach
Unrecorded software update	Unknown treatment implementation

Treatment assignment is persistent state

- ▶ Assignment should occur once according to the experimental protocol.
- ▶ It must persist across sessions and backend restarts.
- ▶ Application code should retrieve the stored assignment rather than recreate it.
- ▶ Treatment access should be enforced server-side.
- ▶ Record the assignment time, protocol version, and relevant randomization information.

Instrumentation is part of measurement

Decide ex ante what the system must record:

- ▶ what a participant saw and when;
- ▶ the content and software version;
- ▶ the selected setting and session context;
- ▶ the participant actions needed for the analysis.

If something matters for inference, log it.

Can you reconstruct the realized data-generating process?

Before launch, ask:

- ▶ Can we reconstruct every treatment exposure?
- ▶ Can we detect impossible states and distinguish no behavior from missing data?
- ▶ Can we identify the software and content version for each exposure?
- ▶ Can we compare realized exposure across treatment groups?
- ▶ Can we connect live-system changes to the observations they affected?

If the system does not record an exposure, the analysis cannot reconstruct it later.

Turn research invariants into technical tests

Invariant	If violated	Test
One treatment per participant	Contamination	Uniqueness test
No customization in control	Contamination	Access and end-to-end test
Event selection remains fixed	Treatment changes	Event-distribution check
Valid version, event, and release linkage	Measurement error	Integrity test

If something matters for identification, test it.

AI as a development tool versus a runtime component

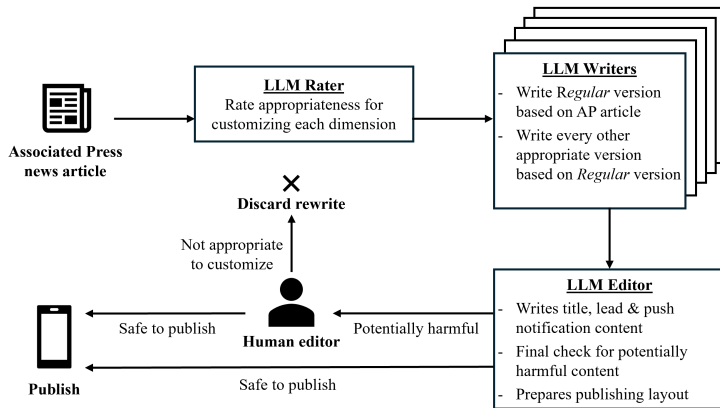
AI as a development tool

- ▶ implementation, debugging, refactoring, and test generation;
- ▶ helps build the system but is not invoked during participant use.

AI as a runtime component

- ▶ generates treatments, evaluates content, or produces user-facing output;
- ▶ becomes part of the live data-generating process.

The production rewrite pipeline



Model calls, automated checks, and human decisions jointly determine what is published.

When AI generates the treatment, what exactly is the treatment?

- ▶ The same prompt can produce different outputs, and providers may update models or moderation behavior.
- ▶ Automated filters and human review affect what participants ultimately see.
- ▶ Record the input, prompt, model version, configuration, output, review decision, and timestamp.
- ▶ Where possible, generate and freeze treatment material before exposure. If generation remains live, preserve every realized output and define the policy that maps inputs into outputs.
- ▶ Analysis and validation of AI-generated treatments is important

Reproducibility requires preserving both the generation process and what participants received.

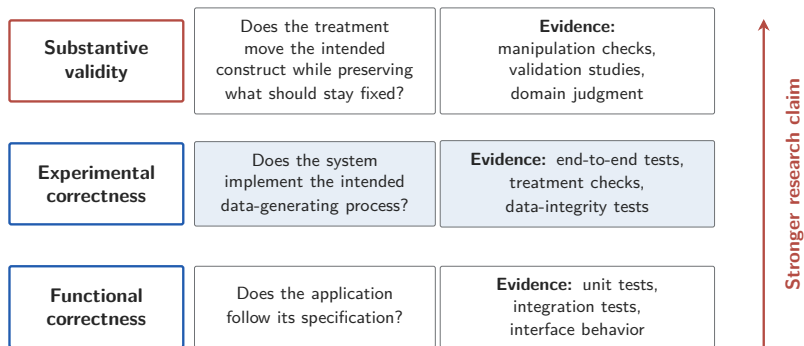
Production systems need a failure and escalation path

Failure	Permitted response
Model timeout	Retry, then use a documented fallback or skip
Output does not parse	Reject and retry
Moderation flag	Send to manual review under a logged rule
Repeated request	Return the existing result

Idempotency: retrying an operation should not create duplicate side effects.

Testing and validating research software

Three kinds of correctness



Each level requires different evidence. Passing one does not establish the next.

Functional correctness uses unit and integration tests

Unit test

- ▶ Given treatment X, return allowed settings Y.

Integration test

- ▶ Selecting Less Complex sends the correct request.
- ▶ The backend retrieves the correct rewrite.
- ▶ The expected application state is updated.

Do the components and their interfaces behave as specified?

Experimental correctness uses end-to-end and integrity tests

End to end

- ▶ Login → assignment → feed → article → setting change → exposure record.

Data integrity

- ▶ exactly one treatment per user;
- ▶ valid version IDs only;
- ▶ no prohibited control exposure;
- ▶ every exposure references a valid user, event, version, and release.

When a bug is fixed, keep a regression test that reproduces it.

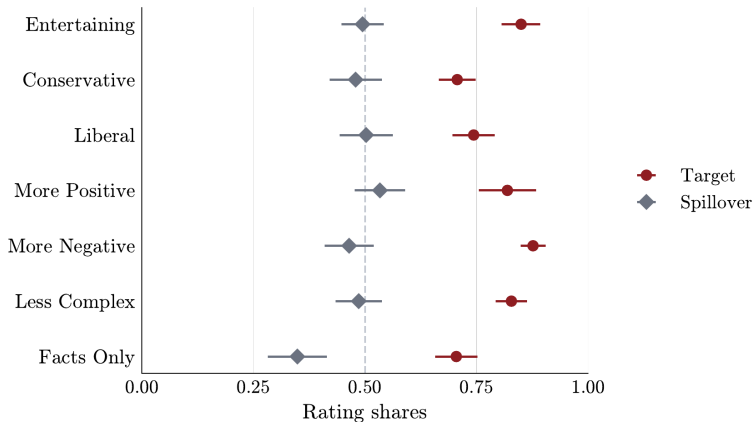
Passing software tests is not enough

A system can pass every engineering test and still manipulate the wrong construct.

The app can perfectly deliver the treatment called “**Less Complex.**”

But did the treatment actually make the article **less complex**?

Does the treatment move the intended construct?



Blinded comparisons separate treatment delivery from treatment validity.

Did what should remain fixed remain fixed?

The treatment changes presentation; the underlying news event should remain constant.

1. Extract factual claims.
2. Work with RAs to check whether the claims are true and consistent across versions.
3. Check numeric information separately.

Applied across **59,092 article versions** (Chopra et al., 2025).

Technical tests do not establish that the rewrite preserved the underlying event.

From localhost to reproducible research

From localhost to production

- ▶ **Development:** local experimentation.
- ▶ **Staging:** production-like testing with non-production data and users.
- ▶ **Production:** real users and real research data.

Once data collection depends on the system, “works on my machine” is no longer a meaningful standard.

Can the software tell you it is failing?

Monitor application errors, API and database failures, model failures, background jobs, and unusual latency.

- ▶ **Research instrumentation:** What happened to participants?
- ▶ **Operational observability:** Is the software working as intended?

Observability asks whether logs, metrics, and errors let you diagnose the health of the system.

Is the study being implemented as planned?

Monitor implementation

During fieldwork

- ▶ assignments and exposure counts by arm;
- ▶ missing events, impossible states, and unexpected versions;
- ▶ errors, crashes, latency, attrition, and delivery failures by treatment.

Protect the research design

Keep separate from outcome analysis

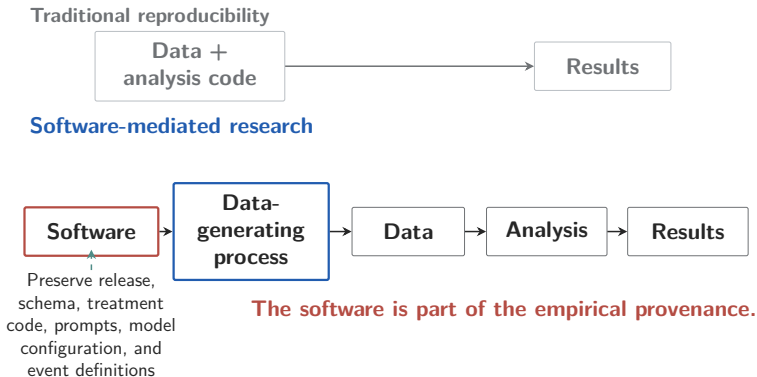
- ▶ use monitoring to diagnose protocol implementation;
- ▶ do not use interim treatment effects to guide unplanned system changes;
- ▶ predefine escalation and documentation procedures where possible.

Changing a live research system safely

Practice	Meaning
Staging	Test changes in a production-like environment
Database migration	Change the schema through a controlled procedure
Feature flag and rollback	Limit a release or restore the previous version
Release and incident log	Record what changed, when, and why

Record which software version generated each observation and document consequential changes during fieldwork.

Reproducibility begins at the data-generating process



Preserve the system that produced the data along with the analysis code.

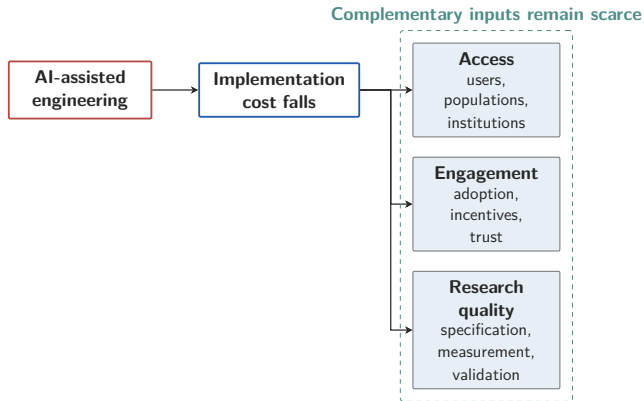
Privacy, security, and ethics constrain the architecture

Ask four questions:

- ▶ What participant information is stored, and for how long?
- ▶ Where does it travel, including to external services and AI providers?
- ▶ Who can access it in development, staging, and production?
- ▶ What do the IRB, consent materials, and data-use agreements permit?

Translate the answers into the architecture: collect less data, separate identifiers where appropriate, keep secrets outside the repository, and restrict access by role and environment.

When engineering gets cheaper, the bottleneck moves



As building gets cheaper, access, participation, measurement, and validation account for more of the remaining difficulty.

A practical workflow for your next research application

1. Start from the **research requirement**.
2. Decide whether to **use, partner, or build**.
3. If building, draw the **architecture and data model**.
4. Prototype quickly.
5. Make consequential **software invariants** explicit.
6. Define **acceptance criteria**.
7. **Test and instrument the system before data collection begins**.
8. Preserve the software state and realized content that generated the data.

Build quickly. Specify explicitly. Test what matters.

References

- ▶ Chopra, Haaland, Roeben, Roth, and Sticher (2025), *News Customization with AI*. CESifo Working Paper 12121.
- ▶ Karpathy (2025), *Vibe Coding*.
- ▶ Sarkar and Drosos (2025), *Vibe Coding: Programming Through Conversation with Artificial Intelligence*.

Further reading

- ▶ Wilson et al. (2014), *Best Practices for Scientific Computing*.
- ▶ Wilson et al. (2017), *Good Enough Practices in Scientific Computing*.
- ▶ Sandve et al. (2013), *Ten Simple Rules for Reproducible Computational Research*.
- ▶ The Turing Way Community, *The Turing Way*.
- ▶ Willison (2025), *Vibe Engineering*.

Questions?